

Microprocessor Systems

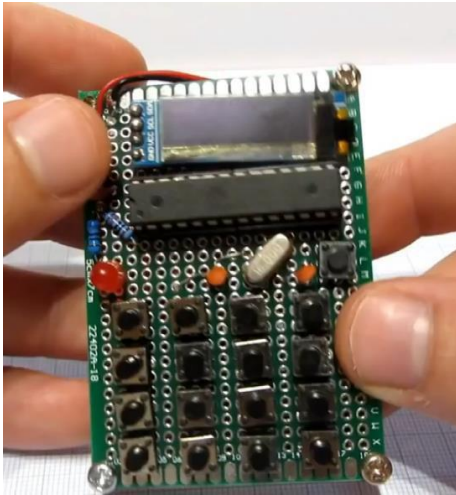
Lab10 – From 8051 to ARC

Prof. Ren-Shuo Liu

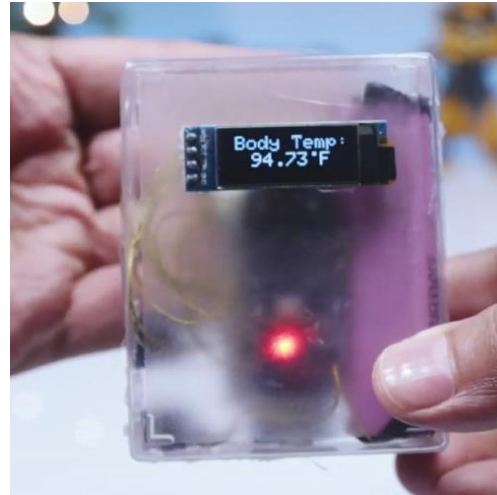
NTHU EE



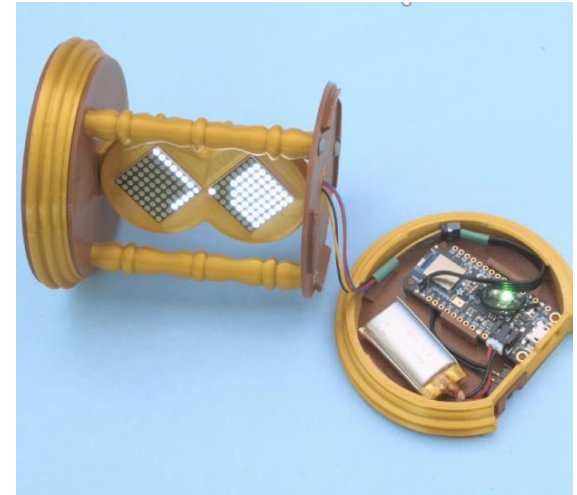
Calculator



IR Thermometer



Digital Hourglass



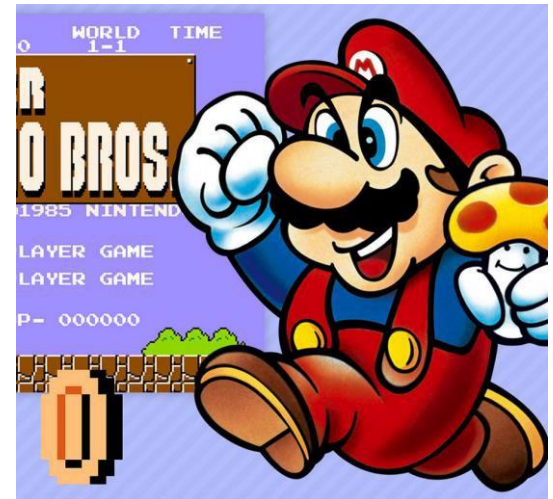
ISP Burner



POV Clock

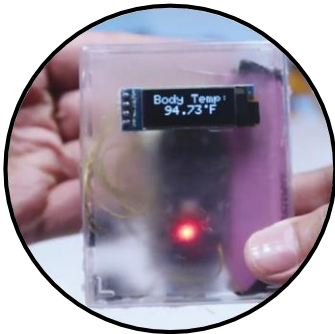


Music Synthesizer

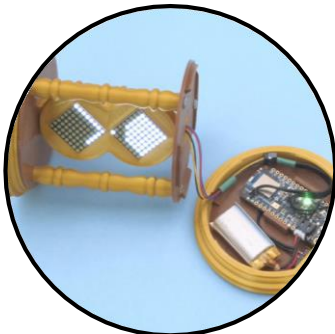




- **GPIO**
- **SPI** interface
- **7-segment** and its driver (MAX7219)
- **Matrix keyboard** scanning



- **I2C** and **SPI** interfaces
- **OLED** and its driver (SSD1305)
- **IR thermal** sensor (MLX90614)
- **IR distance** sensor
- **ADC** (MCP3002)



- **I2C** interface
- 6-Axis **gyroscope** (MPU6050)
- **Matrix LED** and its driver (MAX7219)
- **Timer**



- AT89S51 **ISP** protocol
- Minimal 8051 system
- **Assembly** delay
- Assembly **Interrupt**
- **Mixed C and ASM** programming



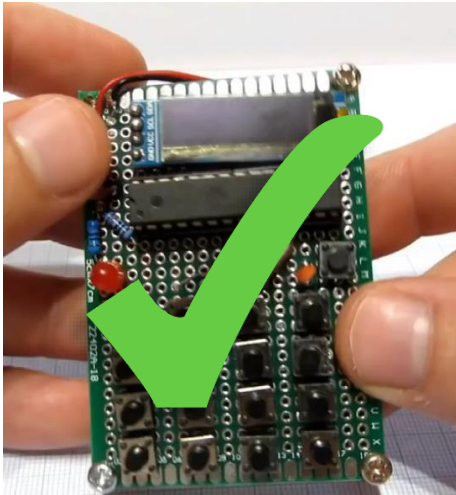
- **Hall** sensor
- **External interrupt**
- Timer
- Software-based LED scanning



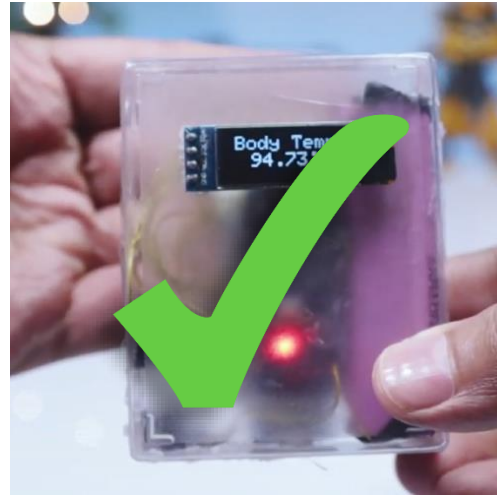
- **Audio synthesis**
- 32-bit controller
- **DAC**



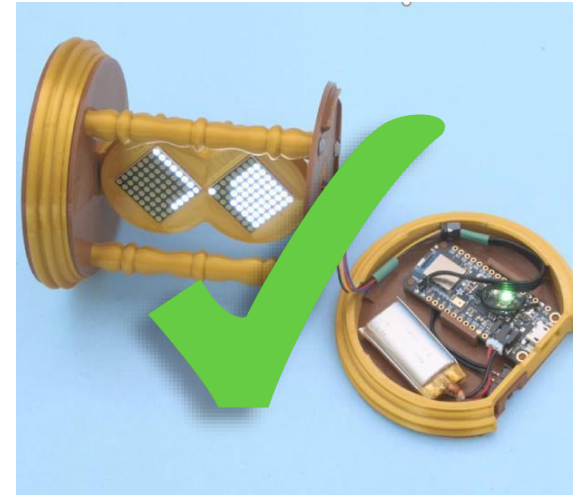
Calculator



IR Thermometer



Digital Hourglass



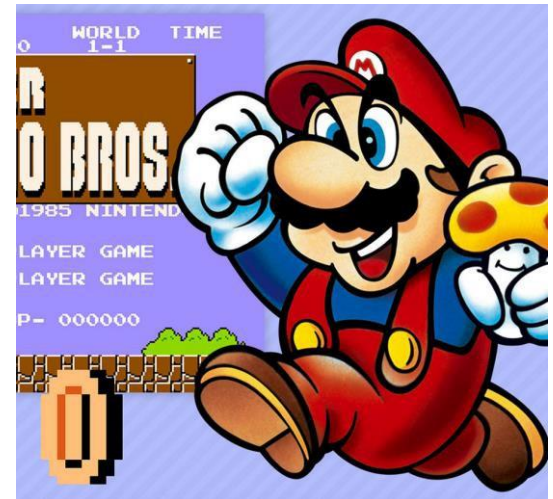
ISP Burner



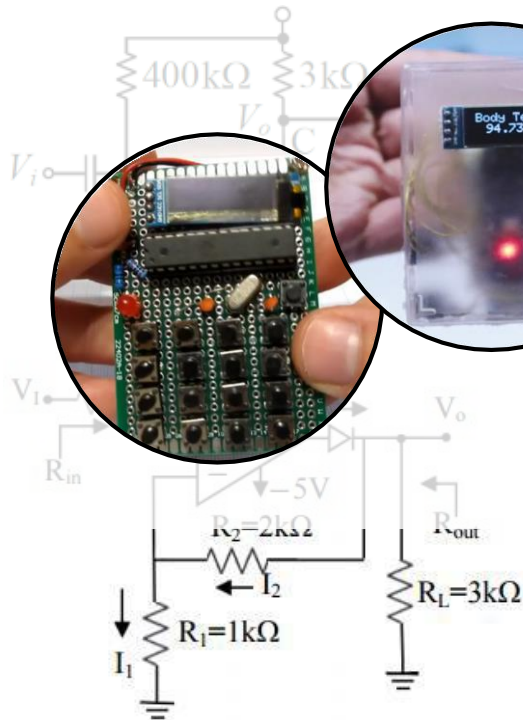
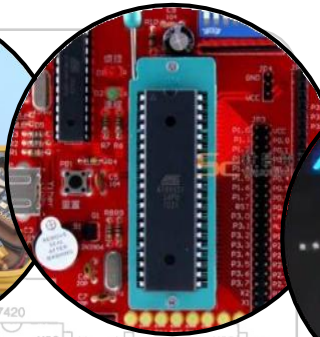
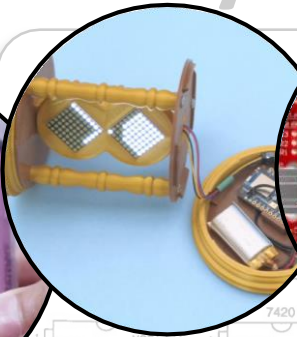
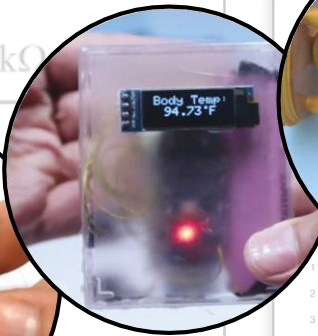
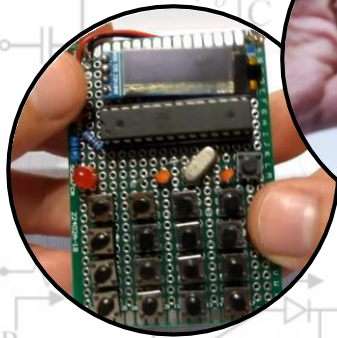
POV Clock



Music Synthesizer



Importance of the Microcontroller and Embedded System Course



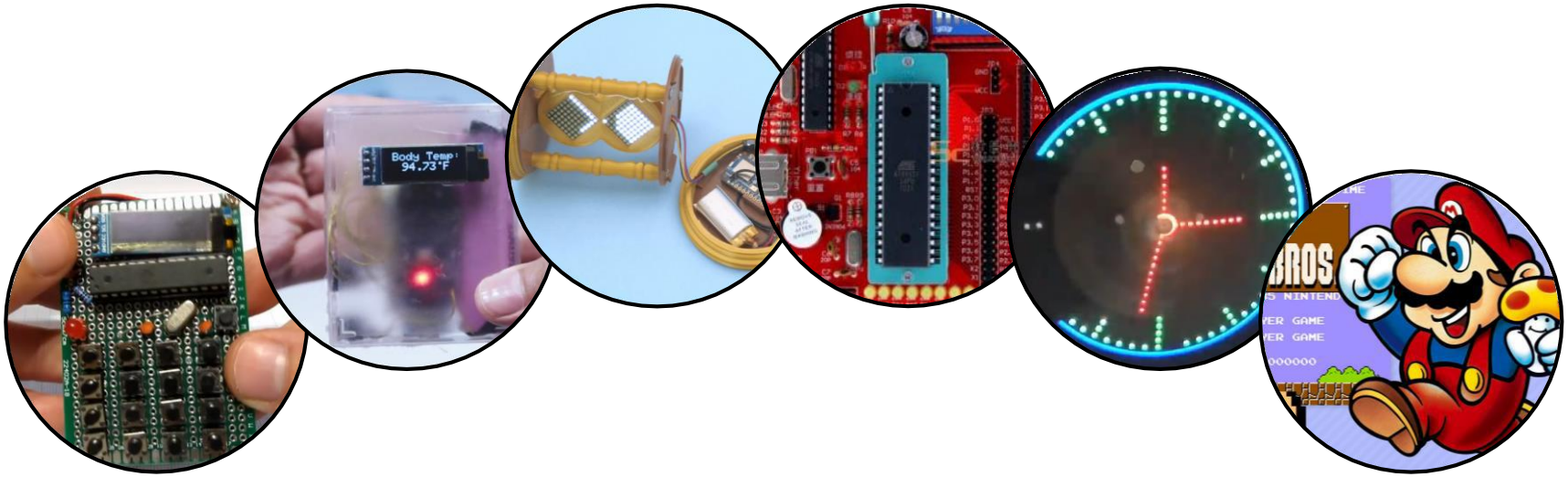
電子學
子路實驗
(Microelectronics)

```
always @ (posedge clk or posedge re
if(reset)
    count <= {size{1'b0}};
else
    begin
        if (count == length - 1)
            count <= {size{1'b0}};
```

邏輯設計
邏設實驗
(Digital Logic Design)

C, C++, Python
程式設計
(Programming)

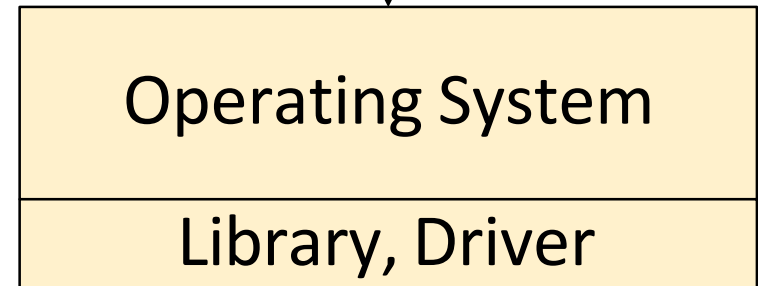
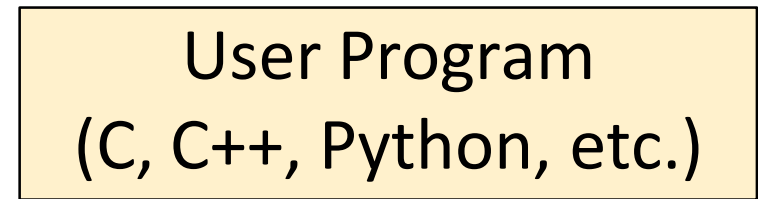
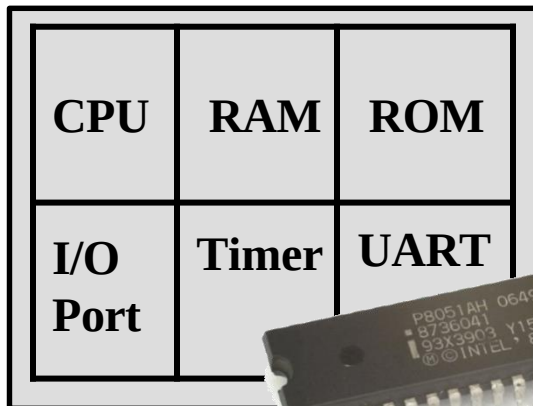
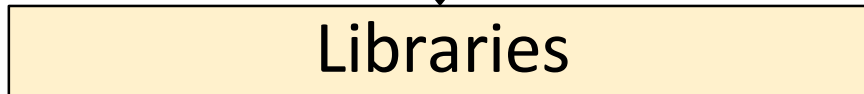
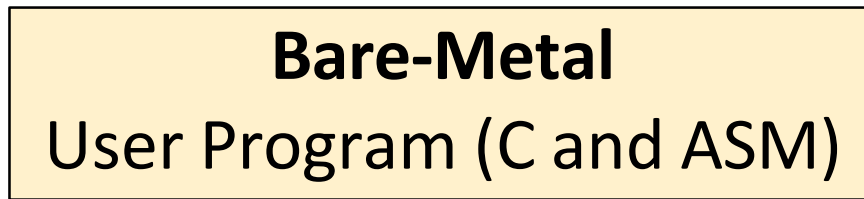
Microcontrollers Such as 8051

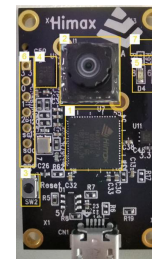


CPU	RAM	ROM
I/O Port	Timer	Serial COM Port



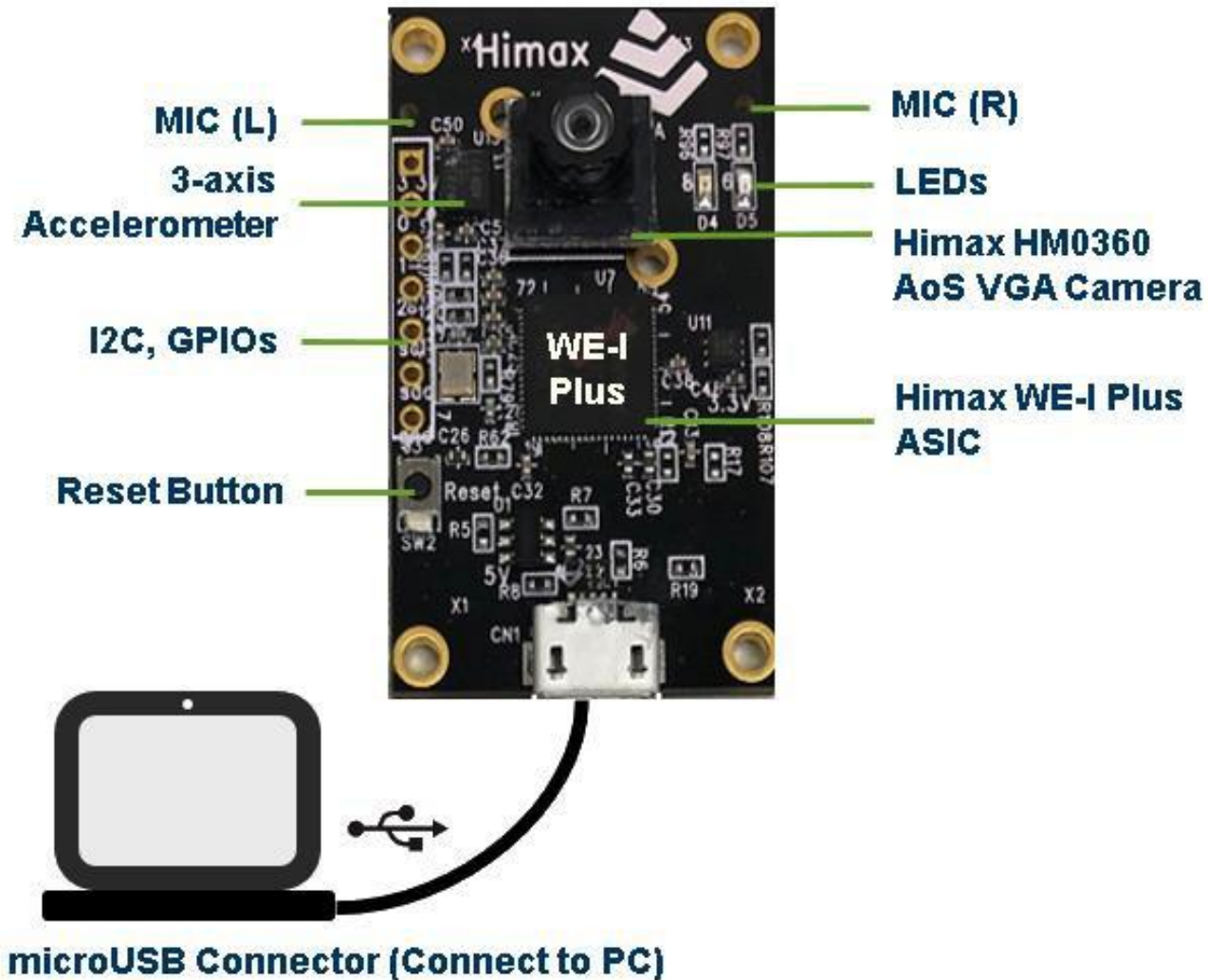
Microcontrollers vs. Common Computers





	AT89S51	Arduino UNO (ATMega328p)	ARC EM	Raspberry 4 (BCM2711)
Pin count	40	28		
ISA	CISC	RISC	RISC	RISC
Word size	8	8	32	64 (4 cores)
Frequency (Hz)	33 M	20 M	400 M	1.5 G
Program Memory	4 K (On-chip)	32 K (On-chip)	2 MB	SD card
Data memory	128 B	2 KB	2 MB	Up to 8 GB (off chip)
GPIO	32	23	3 (44)	Up to 28
Timer	2	3	2	5
PWM channels	0	6	0 (6)	4
ADC channels	0	8	0 (16)	
Serial	UARTx1	USB, UART SPI, I2C	I2Cx1 (USB, UARTx4, SPIx3, I2Cx3, SDIOx2, I2S)	USB, SDIO x2 (I2C or UART) x6 SPI x5, HDMI x2

Himax WE-I Plus

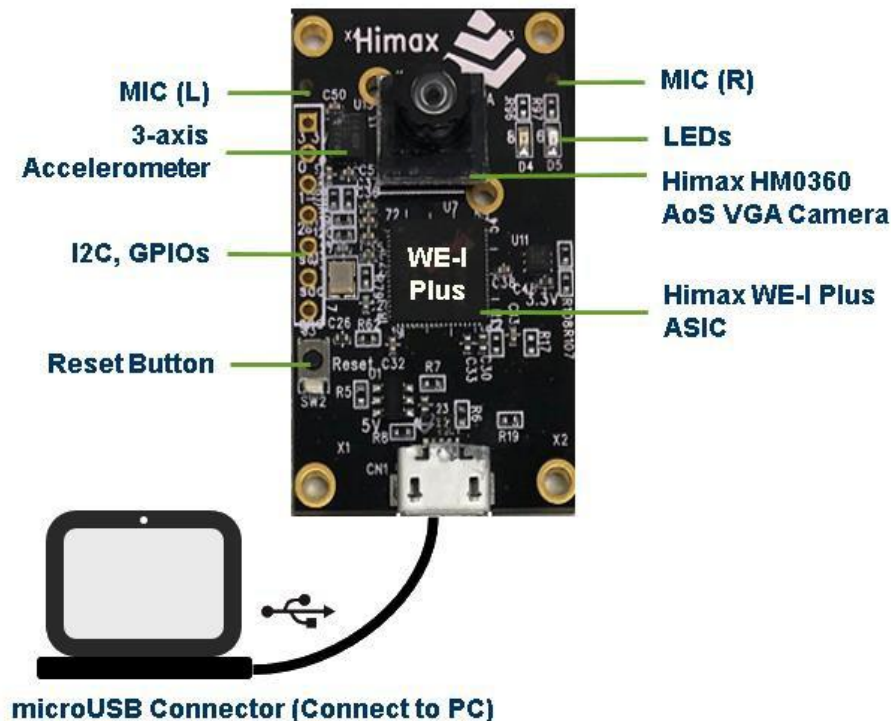


WE-I Features

- CPU: HX6537 (ARC EM9D DSP with FPU)
- Frequency: 400MHz
- SPI program flash: 2MB
- Ram size: 2MB
 - 320kB program ICCM
 - 320kB data DCCM/XCCM/YCCM
 - 1472kB system memory

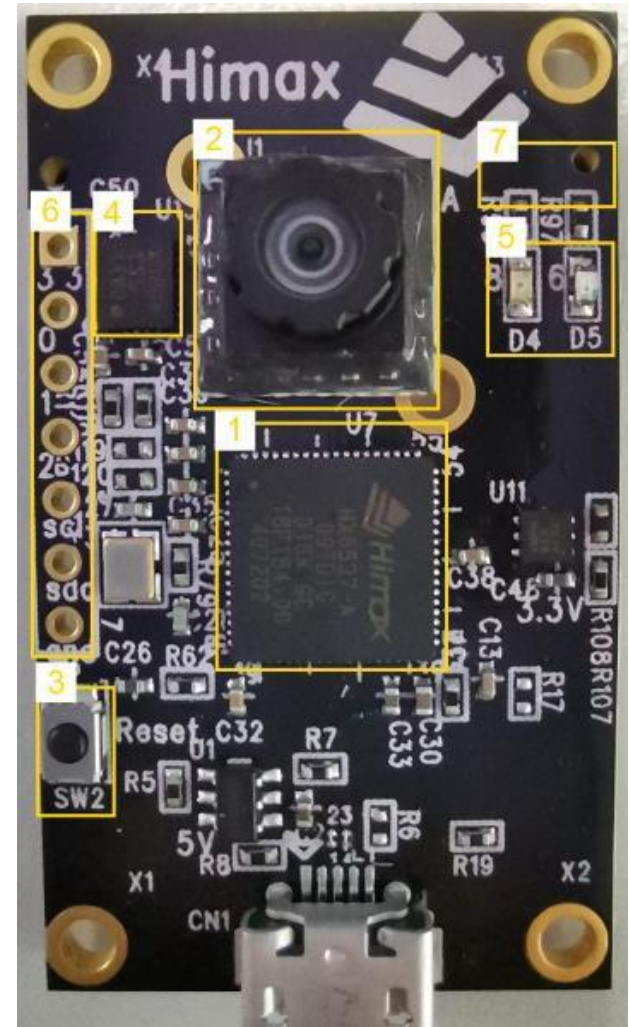
WE-I Features

- UART: 1 (for USB VCP)
- SPI: 1 (for USB VCP)
- I2C Master: 2 (1 for IMU, 1 for user use)
- GPIO: 3



WE-I Features

1. CPU HX6537
2. HM0360 AoSTM VGA camera
Back Side Illuminated (BSI) CMOS
Image Sensor
Active Pixel Array: 656 x 496
Frame Rate: QQVGA 1FPS to VGA
60FPS
3. Reset Button



WE-I Features

4. LSM9DS1 IMU sensor

3 acceleration: ± 2 / ± 4 / ± 8 / ± 16 g

3 angular rate : ± 245 / ± 500 /

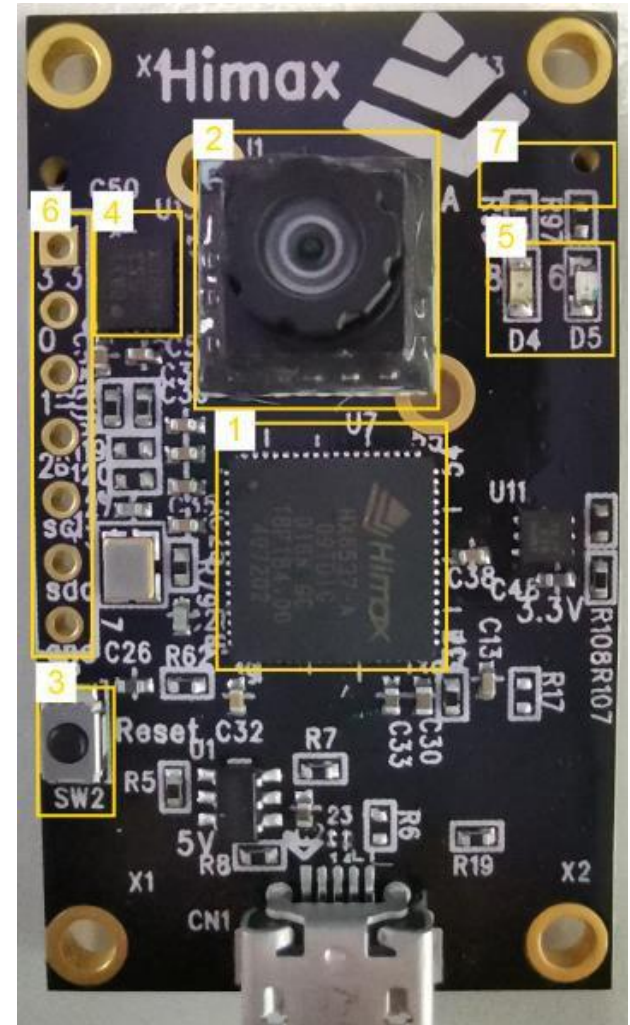
± 2000 dps

3 magnetic field: ± 4 / ± 8 / ± 12 /

± 16 gauss

16-bit data output

5. Green & red LED



WE-I Features

6. Header 7*1 pitch=2.54

Pin1: 3V3

Pin2: GPIO0

Pin3: GPIO1

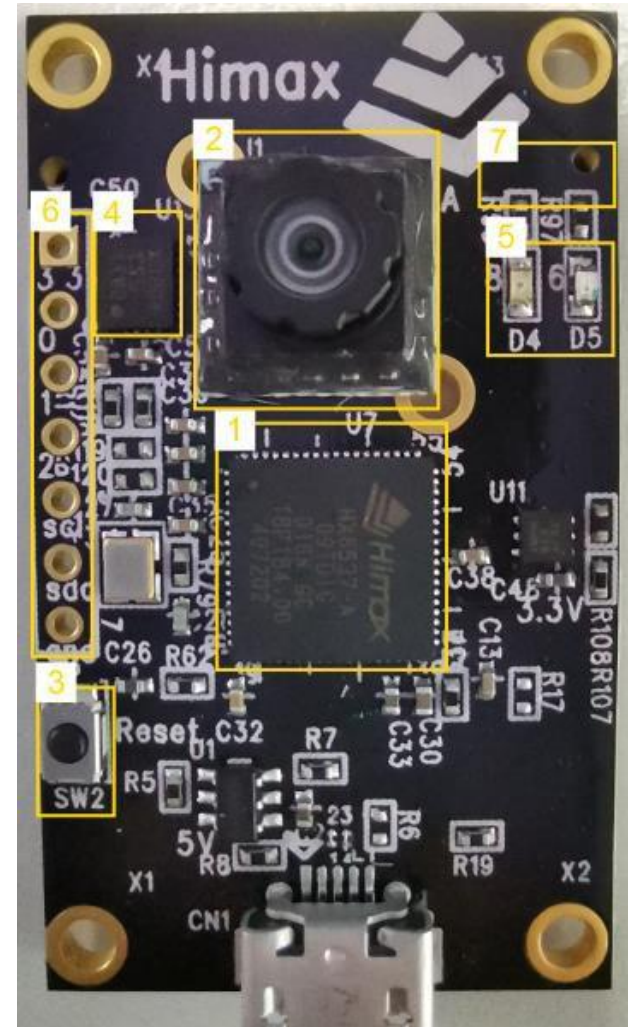
Pin4: GPIO2

Pin5: I2C_M1_SCL

Pin6: I2C_M1_SDA

Pin7: GND

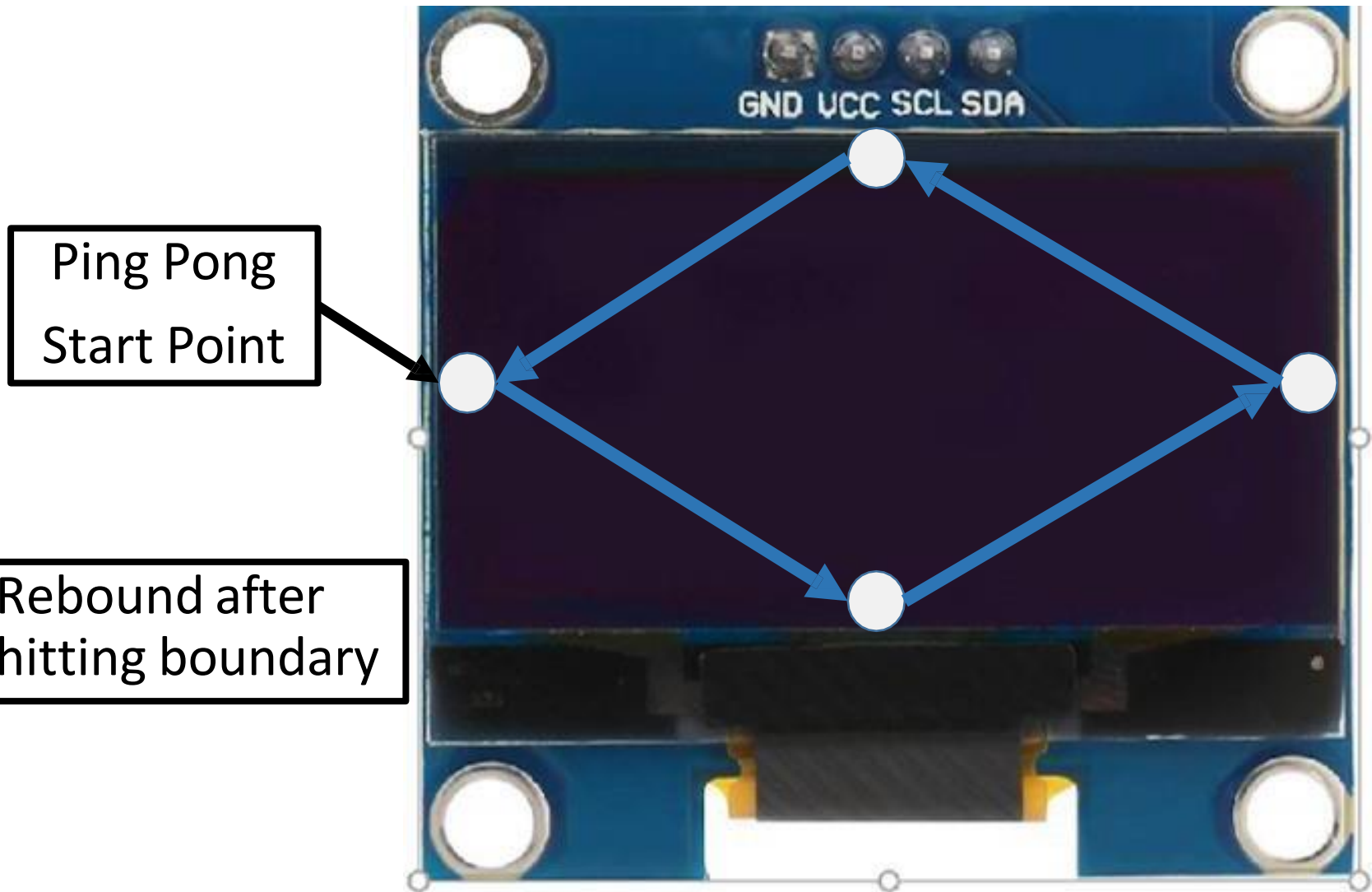
7. Microphones (L/R) at back side



Comparison

	8051 Development Board	WE-I
Processor	AT89S51	EM9D Core
	8-bit	FPU
Supply Voltage (V)	5	3.3
Clock Frequency (MHz)	0 ~ 33	400
Memory (Byte)	FLASH: 4K	FLASH:
	SRAM: 128	SRAM: 2M
UART	X 1	X 1
SPI	none (created by firmware)	1
I2C	none (created by firmware)	2 master

Lab10 on Next Tuesday- Ping Pong Test

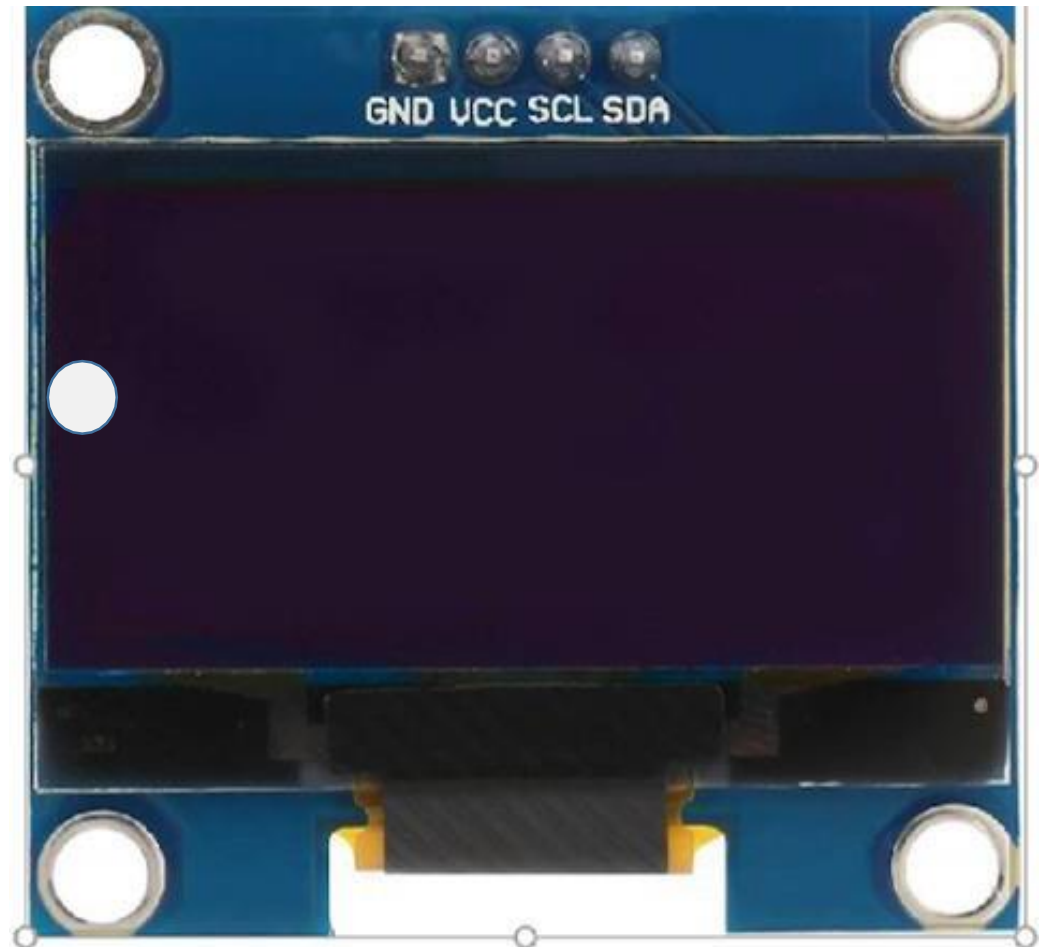


Lab on Next Tuesday- Ping Pong Test

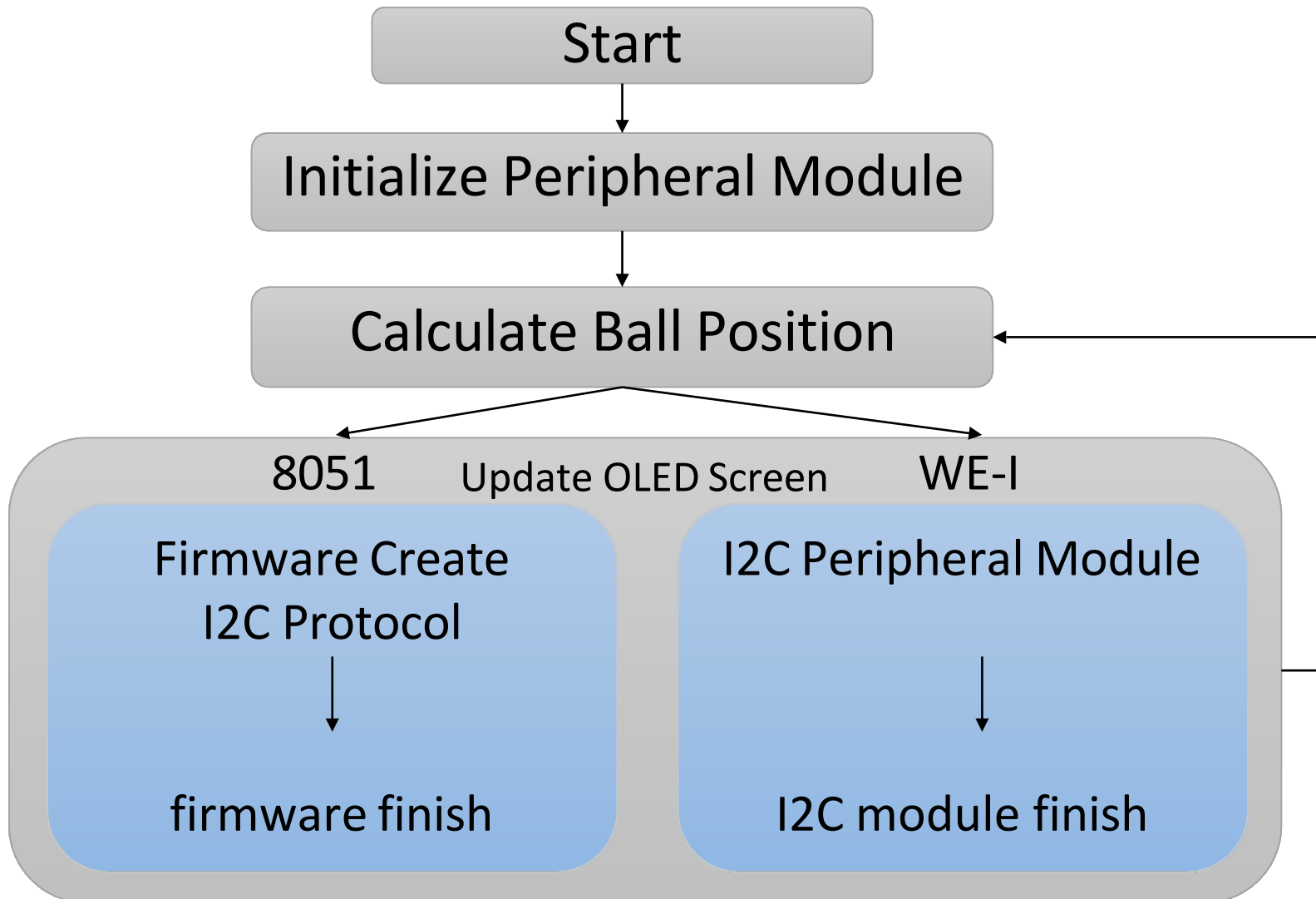
- We will compare the speed of the program between WE-I and 8051 next Tuesday

WE-I vs 8051

Which one is faster?



Lab10 Example Project Procedure



Main Difference

	8051	WE-I
Core	AT89S51	EM9D
I2C	Firmware	Periphe ral Modul e
Calculating	Emulated FP	FPU

Microprocessor Systems

Embedded Operating System

Prof. Ren-Shuo Liu

NTHU EE



Outline

- Operating System (OS)
- Bare-metal / Bare-machine
 - Super-loop programming style
- Key concepts
 - Concurrent tasks (threads)
 - Time multiplexing
 - Preemption
 - User mode vs. kernel mode
 - Real-time OS (RTOS)
 - Mutex and semaphore

Operating Systems

- Software that manages hardware resources to run other software on the hardware
- Examples
 - Windows XP, Windows 7, Windows 8 ...
 - Linux, including the Ubuntu, Fedora, CentOS... distros
 - Android
 - iOS
- Those specifically for embedded systems
 - TinyOS, FreeRTOS, RTLinux, Zephyr...

Q: Which are OSes

- Keil
- Python
- GCC
- Arduino
- Dos
- Microsoft
- TensorFlow

Bare-metal (Bare-machine)

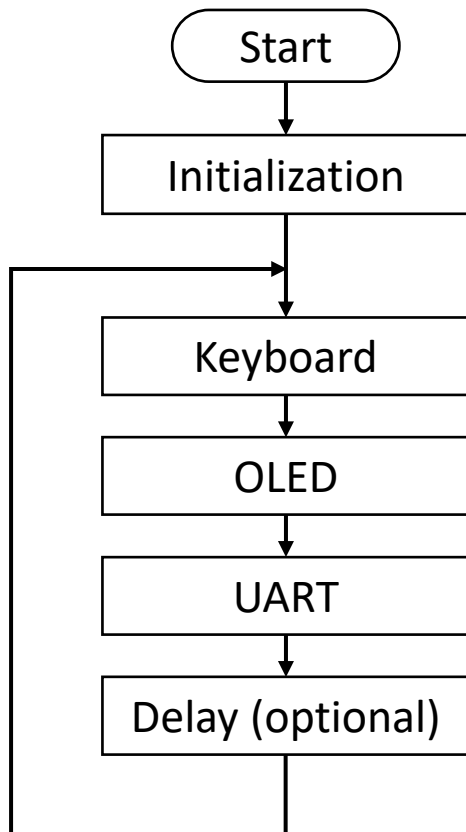
- Developing and directly running software on hardware that has **no OS**
- Benefits
 - Each function owns the entire CPU and occupies interfaces (e.g., UART)
- Drawbacks
 - Difficult to implement a system that concurrently do multiple functions
 - Waste CPU time for slow

Example

- keyboard scanning and de-bouncing + OLED displaying
 - Keyboard may becomes less unresponsive when the microcontroller is drawing the OLED
- Things become complicated if we want more tasks to do in parallel, e.g.,
 - UART communication
 - 6-axis sensing
 - ADC
 - ...

Super Loop

- Super loop is a naïve (and also useful) programming style for bare-metal programming



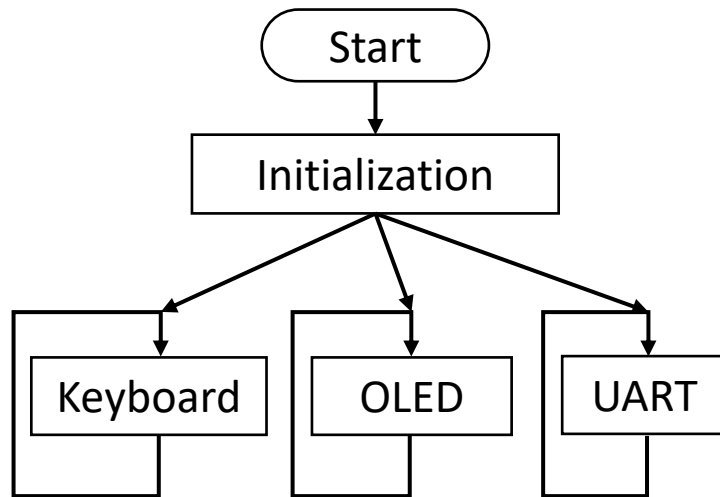
```
int time = 0;
int main()
{
    ...
    for(;;){
        keyboard();
        OLED();
        UART();
        delay(10);
        time++;
    }
    return 0;
}
```

```
void keyboard()
{
    if(time%10==0){ //100 ms
        keyAction();
    }
    return;
}
```

```
void OLED()
{
    if(time%5==0){ //50 ms
        oledAction();
    }
    return;
}
```

Concurrent Tasks

- What if we have multiple CPUs



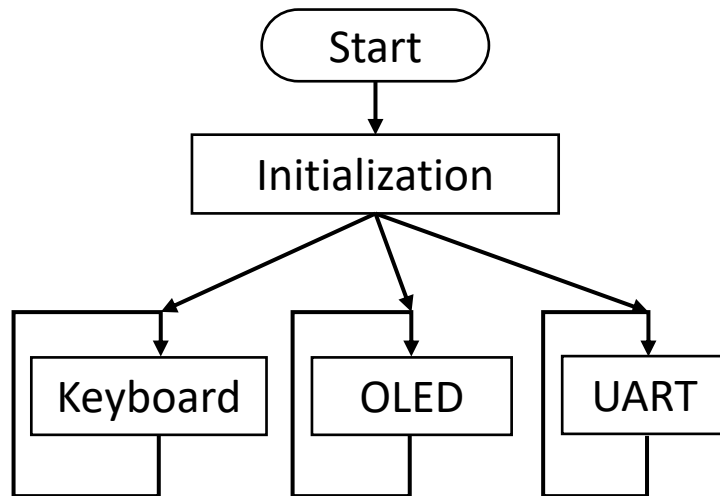
```
int main()
{
    ...
    taskCreate(keyboard);
    taskCreate(OLED);
    taskCrate(UART);
    taskStart();
    while(1);
}
```

```
void keyboard()
{
    while(1){
        keyAction();
        delay(100); //100 ms
    }
    return;
}
```

```
void OLED()
{
    while(1){
        oledAction();
        delay(50); //50 ms
    }
    return;
}
```

Concurrent Tasks

- What if we have **multiple cores** to concurrently do multiple tasks



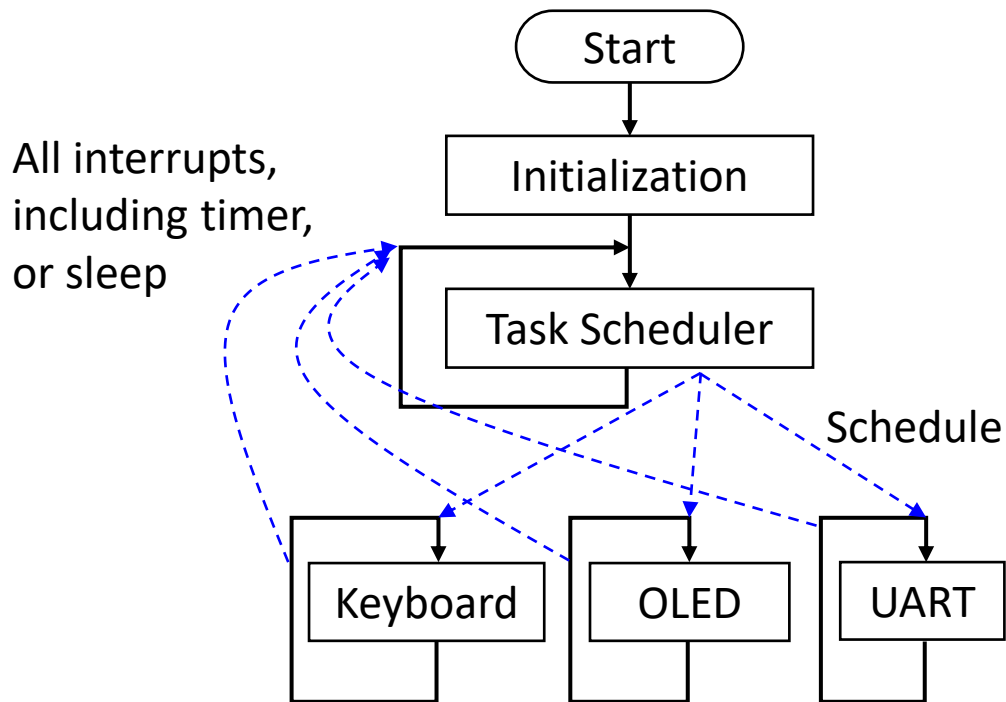
```
int main()
{
    initialization()
    startCore(1, keyboard);
    startCore(2, OLED);
    startCore(3, UART);
    while(1);
}
```

```
void keyboard()
{
    while(1){
        keyAction();
        delay(100); //100 ms
    }
    return;
}
```

```
void OLED()
{
    while(1){
        oledAction();
        delay(50); //50 ms
    }
    return;
}
```

Time Multiplexing

- OS can **virtualize** a core into **multiple cores** by using **time multiplexing** (i.e. time sharing)



```
int main()
{
    ...
    taskCreate(keyboard);
    taskCreate(OLED);
    taskCrate(UART);
    taskStartScheduler();
    while(1);
}
```

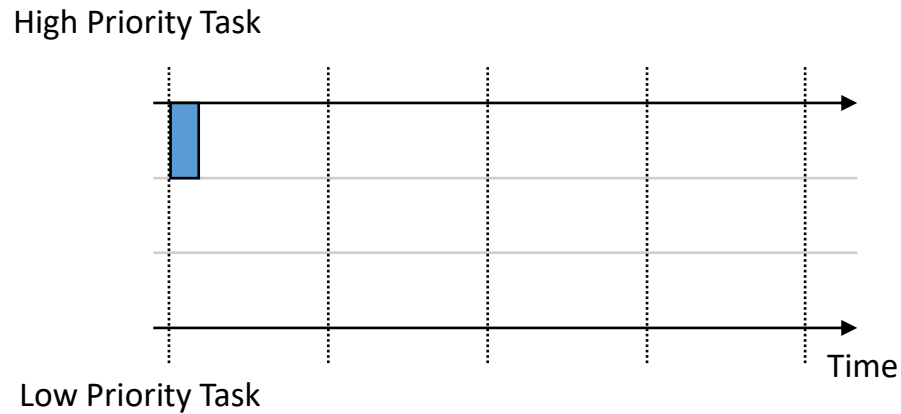
```
void keyboard()
{
    while(1){
        keyAction();
        sleep(100); //100 ms
    }
    return;
}
```

```
void OLED()
{
    while(1){
        oledAction();
        sleep(50); //50 ms
    }
    return;
}
```

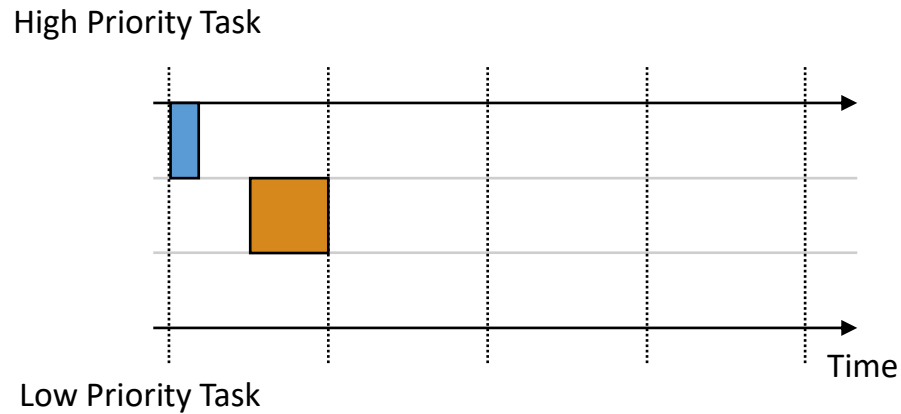
Pre-emptive vs. Non-preemptive OS

- Preemption: 搶佔
- Preemptive OS has the right to suspend tasks
 - For time sharing (timer interrupt)
 - For prioritizing other tasks (e.g., external interrupt)
- In a non-preemptive OS, a task can occupy CPUs as long as it wishes
 - OS does not force tasks to relinquish the CPU
 - Tasks have to co-operate by themselves
 - This is not the case for Windows, Linux, Android. Otherwise, a buggy user program can occupy the computer

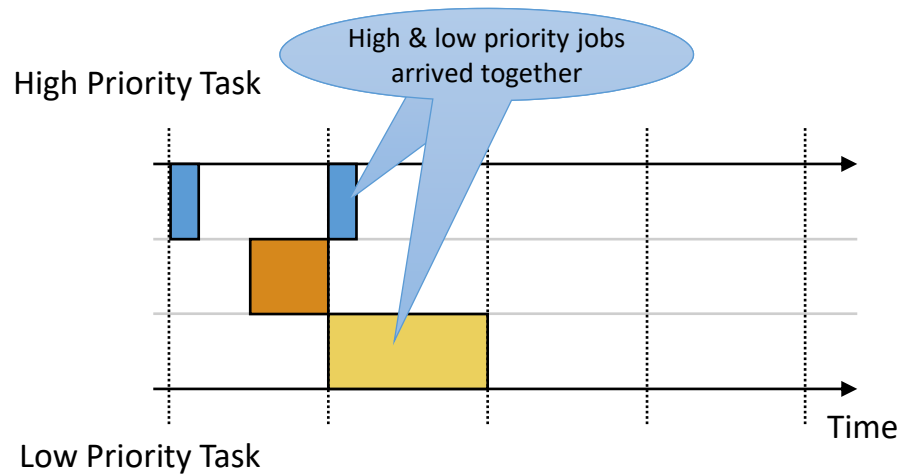
Preemptive Fixed Priority Scheduling



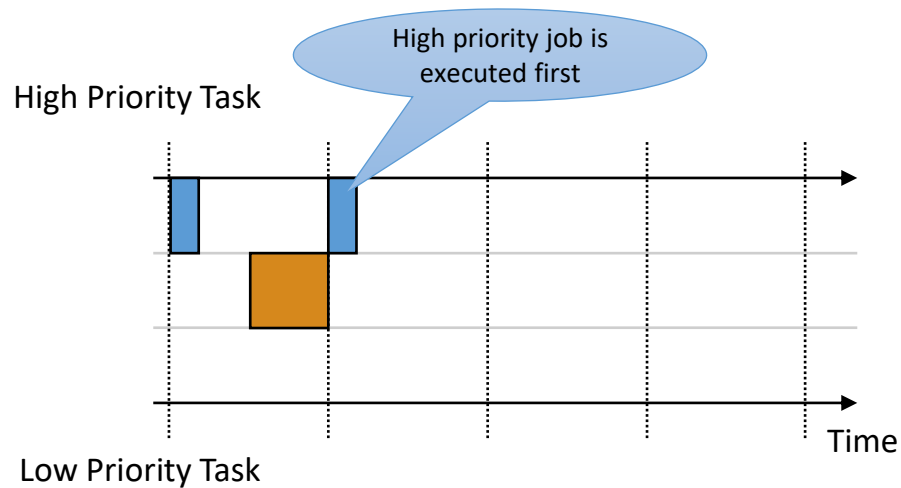
Preemptive Fixed Priority Scheduling



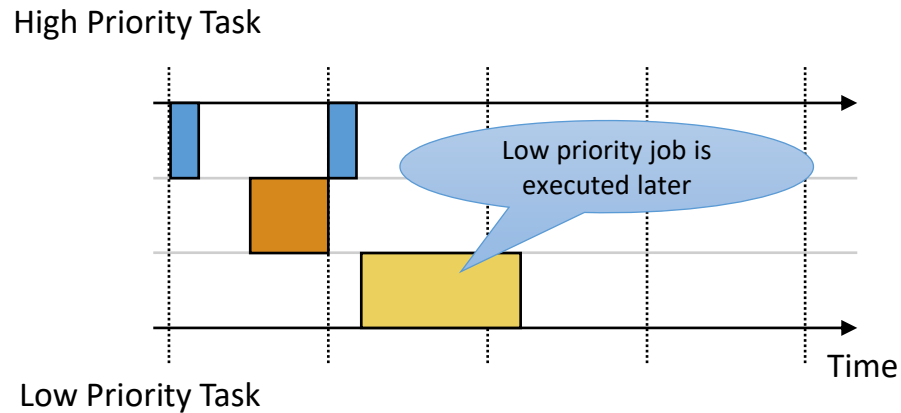
Preemptive Fixed Priority Scheduling



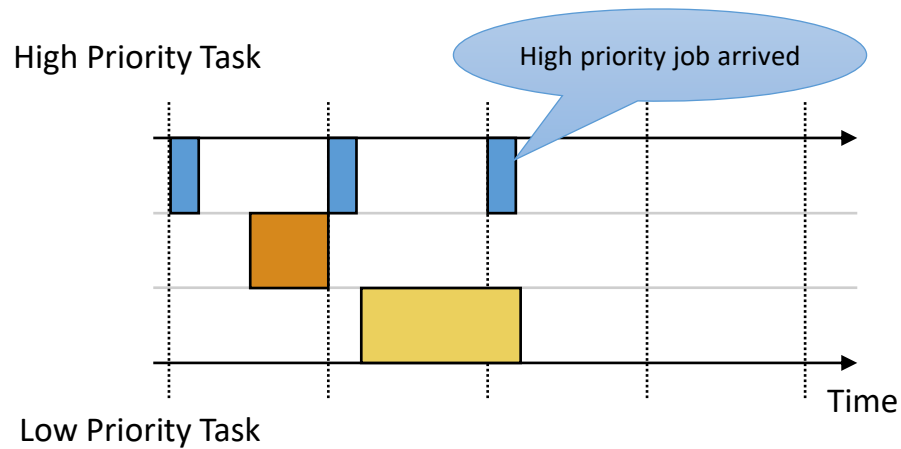
Preemptive Fixed Priority Scheduling



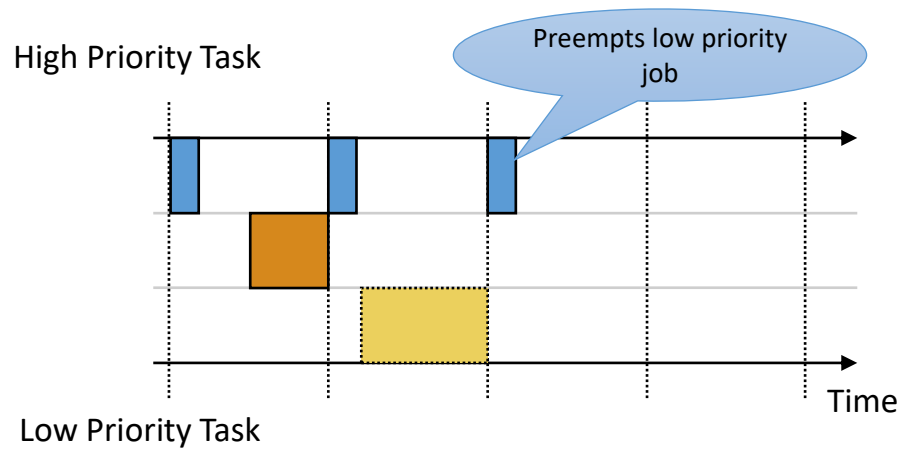
Preemptive Fixed Priority Scheduling



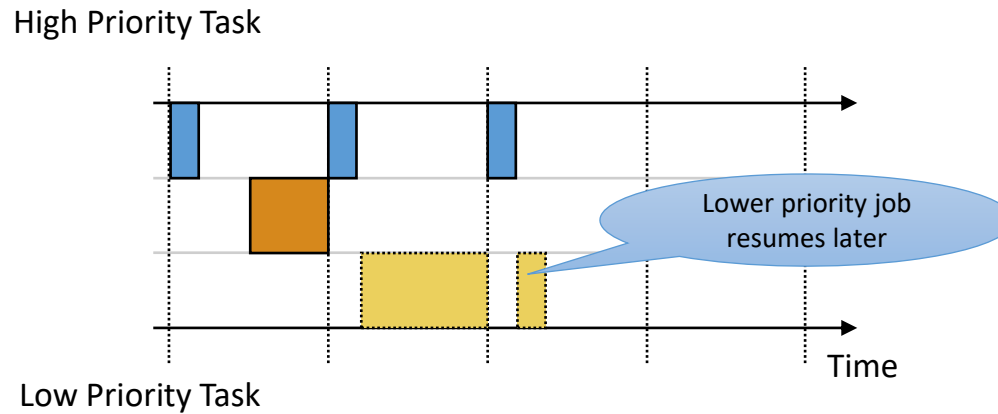
Preemptive Fixed Priority Scheduling



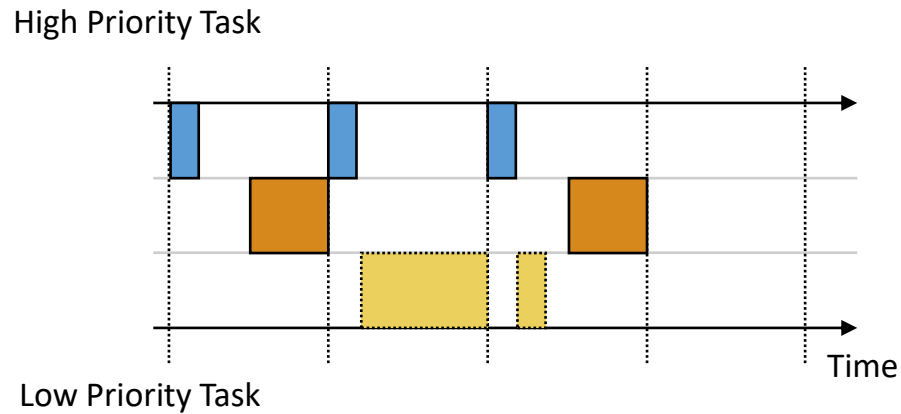
Preemptive Fixed Priority Scheduling



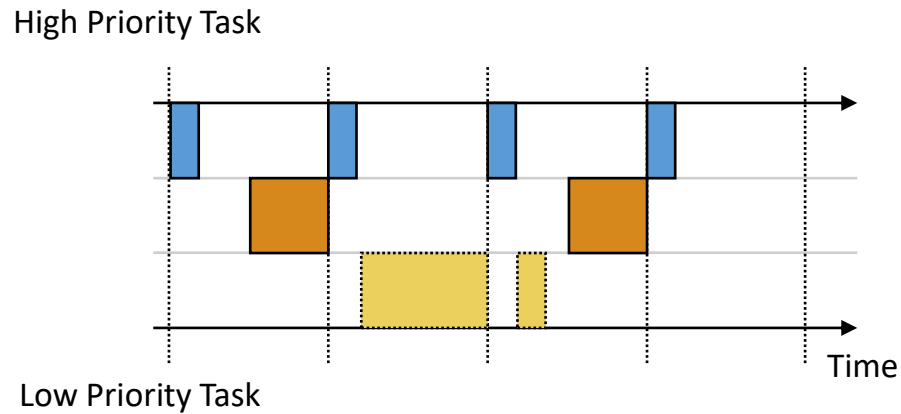
Preemptive Fixed Priority Scheduling



Preemptive Fixed Priority Scheduling



Preemptive Fixed Priority Scheduling



User Mode vs. Kernel Mode

- What happens if a task disables interrupts in a pre-emptive OS?
 - e.g., EA=0
 - Can the OS still force the task to relinquish the CPU?
- Clearly, the interrupt switches (and many other things, of course) should only be alterable by the OS only rather than the tasks
 - CPU hardware enforces that only the OS can alter them

User Mode vs. Kernel Mode

- How can CPU hardware distinguish OS from other tasks?
 - CPU is designed to be in the **kernel mode** after a reset
 - First piece of code running on the CPU should be the OS
 - OS always direct interrupts to itself
 - Kernel mode flag is set upon interrupts
 - OS clears the kernel mode flag before it schedules a task
- Q: Does 8051 support user/kernel mode?
 - 8051 does not have the user/kernel mode design

Real-Time OS

- OS helps to time-multiplex tasks
- Some applications need time-related guarantees
 - Audio/Video player
 - Automotive computer
 - Flight control
 - Weapon control
 - Medical equipment
 - Nuclear power plant controller
- Q: Are Windows and Linux real-time OS?
 - They are not
 - e.g., user may experience “正為您更新作業系統... 5%”

Real-Time OS

- Some embedded OSes are designed to offer real-time guarantees
 - RTLinux, FreeRTOS, etc.
- Real-time levels
 - Hard real-time
 - Violating time guarantees is not acceptable
 - Soft real-time
 - Occasionally violating time guarantees is acceptable
e.g., video player

Mutex and Semaphore

- Both are synchronization mechanisms among tasks
- Handle situation that multiple tasks share a limited number of resources, e.g., UART
 - Without synchronization, UART outputs from two tasks may mixed together because of time sharing
- Is the following implementation correct?

```
bool busy = false;
int main()
{
    ...
    taskCreate(task1);
    taskCreate(task2);
    taskStartScheduler();
    while(1);
}
```

```
void task1()
{
    while(busy==true);
    busy = true;

    uartput("1");
    uartput("2");
    uartput("3");
    return;
}
```

```
void task2()
{
    while(busy==true);
    busy = true;

    uartput("A");
    uartput("B");
    uartput("C");
    return;
}
```